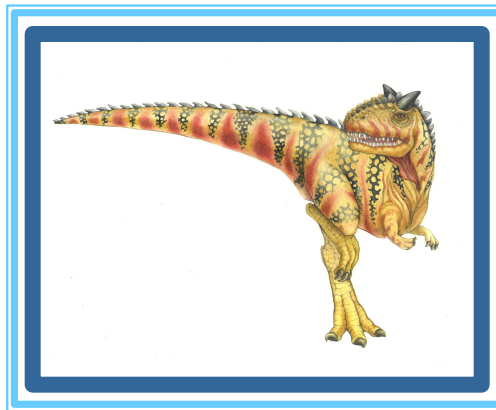


Process Scheduling

Rakhshan Imtiaz





Basic

Concepts

- In multiprogramming systems, where there is more than one process runnable (ready), the OS must decide which one to run next
- We already talked about the **dispatcher**, whose job is to allow the next process to run
- It's intimately associated with a **scheduler** whose job is to decide (using a **scheduling algorithm**) which process to have the dispatcher dispatch
- We will focus on the scheduling, understand the problems associated with it, and look at some scheduling algorithms





Schedule

rs

- Chooses a ready process and activates it
- Policy – which process to choose
- Choice of policy depends on goals:
 - Maximize throughput – avoid idle time and overhead
 - Fairness – all jobs get same service
 - Minimum average waiting time – good service to some
 - Good service to “important” jobs





Schedule rs

- Short term Scheduler or CPU Scheduling
- Long term Scheduler or Job Scheduler
- Medium Term Scheduler





Schedule rs

- **Short term Scheduler or CPU Scheduling**
 - Which program is to be run next
- **Long term Scheduler or Job Scheduler**
 - Which ready jobs should be brought to memory
 - May need to invoke only when a process leaves the system
 - Must make a careful selection





Schedule

rs

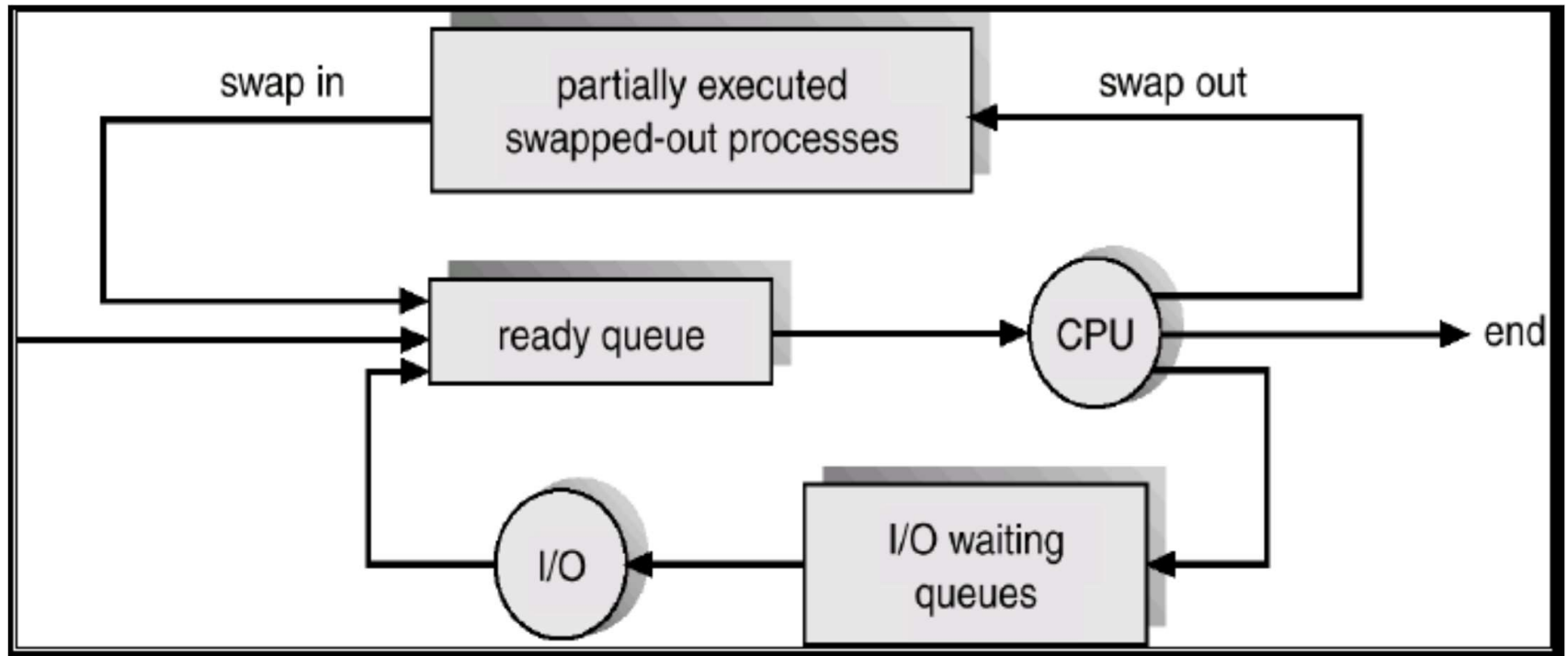
- Sometimes OS may swap a blocked process to disk to free up more memory
- Or to improve process mix
- This is also called **Swapping**
- This scheduler responsible for this task is known as

Intermediate or Medium Term Scheduler





Addition of Medium Term Scheduling





Schedulers

(Cont.)

- Short-term scheduler is invoked very frequently (milliseconds) ⇒ Design Objective? **(must be fast)**
- Long-term scheduler is invoked very (seconds, minutes) ⇒ **(may be** infrequently **slower)**
- The long-term scheduler controls the degree of *multiprogramming*





Non Preemptive Scheduling

- Once the CPU has been allocated to a process
- The process keeps it until
 - It Terminates
- There is no way, to get the CPU back, FORCEFULLY





Preemptive Scheduling

- Possible to get the CPU back from a process, even if the process has not completed its execution.





Scheduling Criteria

- CPU Utilization
 - Keep the CPU as busy as is possible
 - May range from 0% to 100%
- Throughput
 - Number of processes completed per unit time
- Turnaround Time
 - How long it take to execute a Process

**Turnaround = Completion_Time
- Submission_Time**





Scheduling Criteria

- Waiting Time
 - Sum of the periods spent waiting in the Ready Queue

$$W.T = \text{Finish Time} - \text{Burst Time} - \text{Arrival Time}$$

- Response Time
- **Response_Time =
First_Response_Start_Time
- Submission_Time**





Optimization Criteria

- We would like to **Maximize**
 - CPU Utilization
 - Throughput
- And **Minimize**
 - Turnaround Time
 - Waiting Time
 - Response Time





Scheduling Algorithms

- First come, First serve (FCFS)
- Shortest Job First (SJF)
- Priority Scheduling
- Round-Robin Scheduling



END

